

15.071 — Deliverable 1

Cambridge Computers: Predicting Laptop Price

Problem 4 — Logistic Regression

Management now wants to classify a laptop as **high-priced** (`Price >= 500`) or not.

Problem 4 setup: create the binary outcome `high`

`high = 1` if `Price >= 500` Euros, `high = 0` otherwise, in **both** train and test.

```
In [14]: train["high"] = (train["Price"] >= 500).astype(int)
test["high"] = (test["Price"] >= 500).astype(int)

print("Train high counts:\n", train["high"].value_counts().sort_index().to_string())
print("\nTest high counts:\n", test["high"].value_counts().sort_index().to_string())
print(f"\nProportion high -- train: {train['high'].mean():.4f} | test: {test['high'].mean():.4f}")
```

Train high counts:

	high
0	115
1	550

Test high counts:

	high
0	53
1	227

Proportion high -- train: 0.8271 | test: 0.8107

Problem 4(a)

Logistic regression predicting `high` from all independent variables in Table 1. **No variable selection is performed.**

```
In [15]: log_formula = "high ~ C(Company) + C(TypeName) + C(GPU) + Screen + Memory + Weight + Rating"
log_model = smf.logit(log_formula, data=train).fit()
print(log_model.summary())
```

Warning: Maximum number of iterations has been exceeded.
 Current function value: 0.251696
 Iterations: 35

Logit Regression Results

Dep. Variable:	high	No. Observations:	665
Model:	Logit	Df Residuals:	653
Method:	MLE	Df Model:	11
Date:	Tue, 22 Sep 2026	Pseudo R-squ.:	0.4534
Time:	17:09:49	Log-Likelihood:	-167.38
converged:	False	LL-Null:	-306.24
Covariance Type:	nonrobust	LLR p-value:	4.281e-53

	coef	std err	z	P> z	[0.025	0.975]
Intercept	29.4861	5959.094	0.005	0.996	-1.17e+04	1.17e+04
C(Company) [T.Dell]	1.2456	0.501	2.486	0.013	0.263	2.228
C(Company) [T.HP]	1.4910	0.465	3.206	0.001	0.579	2.403
C(Company) [T.Lenovo]	-0.0525	0.445	-0.118	0.906	-0.925	0.820
C(TypeName) [T.Notebook]	-17.6031	5959.093	-0.003	0.998	-1.17e+04	1.17e+04
C(TypeName) [T.Ultrabook]	-16.1946	5959.093	-0.003	0.998	-1.17e+04	1.17e+04
C(GPU) [T.Intel]	-0.1053	0.354	-0.298	0.766	-0.799	0.589
C(GPU) [T.Nvidia]	2.0148	0.605	3.329	0.001	0.829	3.201
Screen	-1.1954	0.311	-3.845	0.000	-1.805	-0.586
Memory	0.7421	0.093	8.019	0.000	0.561	0.923
Weight	1.5361	0.903	1.701	0.089	-0.234	3.306
Rating	-0.0904	0.049	-1.838	0.066	-0.187	0.006

Possibly complete quasi-separation: A fraction 0.20 of observations can be perfectly predicted. This might indicate that there is complete quasi-separation. In this case some parameters will not be identified.

/Users/chrislowzx/miniconda3/lib/python3.13/site-packages/statsmodels/base/model.py:607: ConvergenceWarning: Maximum Likelihood optimization failed to converge. Check mle_retvals
 warnings.warn("Maximum Likelihood optimization failed to "

Answer (a). The fitted logistic regression model using all independent variables is shown above. The model produces a convergence warning due to all Gaming laptops in the training data are high-priced. So some TypeName

coefficient estimates should be interpreted with caution.

Problem 4(b)

Which variables are significant in predicting the probability of a high price?

```
In [17]: sig = pd.DataFrame({"coef": log_model.params, "p_value": log_model.pvalues})
sig["signif_5pct"] = np.where(sig["p_value"] < 0.05, "YES", "no")
print(sig.round(4).to_string())

print("\nSignificant at 5%:", [v for v in sig.index[sig["p_value"] < 0.05] if v != "Intercept"])
```

	coef	p_value	signif_5pct
Intercept	29.4861	0.9961	no
C(Company) [T.Dell]	1.2456	0.0129	YES
C(Company) [T.HP]	1.4910	0.0013	YES
C(Company) [T.Lenovo]	-0.0525	0.9061	no
C(TypeName) [T.Notebook]	-17.6031	0.9976	no
C(TypeName) [T.Ultrabook]	-16.1946	0.9978	no
C(GPU) [T.Intel]	-0.1053	0.7661	no
C(GPU) [T.Nvidia]	2.0148	0.0009	YES
Screen	-1.1954	0.0001	YES
Memory	0.7421	0.0000	YES
Weight	1.5361	0.0889	no
Rating	-0.0904	0.0660	no

Significant at 5%: ['C(Company) [T.Dell]', 'C(Company) [T.HP]', 'C(GPU) [T.Nvidia]', 'Screen', 'Memory']

Answer (b). At the 5% level the significant predictors are **Memory** ($p < 0.001$), **Screen** ($p < 0.001$), **GPU = Nvidia** ($p = 0.001$), and **Company = HP** ($p = 0.001$) and **Company = Dell** ($p = 0.013$) relative to the Asus baseline. **Weight** ($p = 0.089$) and **Rating** ($p = 0.066$) are significant only at the 10% level, and **GPU = Intel** and **Company = Lenovo** are not significant.

Note that the **TypeName** p-values (≈ 0.998) are meaningless here because of the separation described in part (a).

Problem 4(c)

Are the significant variables the same as in the Problem 1 linear model?

```
In [18]: compare = pd.DataFrame({
    "linear_coef": lin_model.params,
    "linear_p":    lin_model.pvalues,
    "logit_coef":  log_model.params,
    "logit_p":     log_model.pvalues,
})
compare["linear_sig_5pct"] = np.where(compare["linear_p"] < 0.05, "YES", "no")
compare["logit_sig_5pct"]  = np.where(compare["logit_p"]  < 0.05, "YES", "no")
compare["same_significance"] = np.where(
    compare["linear_sig_5pct"] == compare["logit_sig_5pct"], "same", "DIFFERENT")
print(compare.round(4).to_string())
```

	linear_coef	linear_p	logit_coef	logit_p	linear_sig_5pct	logit_sig_5pct
same_significance						
Intercept	1434.2466	0.0000	29.4861	0.9961	YES	no
DIFFERENT						
C(Company) [T.Dell]	86.7254	0.0438	1.2456	0.0129	YES	YES
same						
C(Company) [T.HP]	170.3954	0.0001	1.4910	0.0013	YES	YES
same						
C(Company) [T.Lenovo]	35.3279	0.4020	-0.0525	0.9061	no	no
same						
C(TypeName) [T.Notebook]	-76.2954	0.1943	-17.6031	0.9976	no	no
same						
C(TypeName) [T.Ultrabook]	416.3939	0.0000	-16.1946	0.9978	YES	no
DIFFERENT						
C(GPU) [T.Intel]	165.4297	0.0000	-0.1053	0.7661	YES	no
DIFFERENT						
C(GPU) [T.Nvidia]	218.1386	0.0000	2.0148	0.0009	YES	YES
same						
Screen	-120.9323	0.0000	-1.1954	0.0001	YES	YES
same						
Memory	87.3725	0.0000	0.7421	0.0000	YES	YES
same						
Weight	271.0986	0.0000	1.5361	0.0889	YES	no
DIFFERENT						
Rating	-11.8206	0.0106	-0.0904	0.0660	YES	no
DIFFERENT						

Answer (c). The significant predictors are not identical between the two models. **Memory, Screen, GPU = Nvidia, Company = Dell, and Company = HP** are significant in both, while **GPU = Intel, Weight, Rating, and TypeName = Ultrabook** are significant only in the linear model.

Problem 4(d)

For each significant variable, does an increase in its value raise or lower the probability of being high-priced?

```
In [19]: signif = compare.loc[(compare["logit_p"] < 0.05) & (compare.index != "Intercept")].copy()
signif["odds_ratio"] = np.exp(signif["logit_coef"])
```

```
signif["direction"] = np.where(signif["logit_coef"] > 0, "INCREASES P(high)", "DECREASES P(high)")
print(signif[["logit_coef", "odds_ratio", "logit_p", "direction"]].round(4).to_string())
```

	logit_coef	odds_ratio	logit_p	direction
C(Company) [T.Dell]	1.2456	3.4750	0.0129	INCREASES P(high)
C(Company) [T.HP]	1.4910	4.4416	0.0013	INCREASES P(high)
C(GPU) [T.Nvidia]	2.0148	7.4991	0.0009	INCREASES P(high)
Screen	-1.1954	0.3026	0.0001	DECREASES P(high)
Memory	0.7421	2.1003	0.0000	INCREASES P(high)

Answer (d).

Among the significant predictors, higher **Memory** increases the probability of being high-priced (coefficient = 0.742, odds ratio = 2.10), which makes sense since additional RAM is generally associated with more expensive laptops.

Nvidia GPUs (coefficient = 2.015, OR = 7.50), **HP** (1.491, OR = 4.44), and **Dell** (1.246, OR = 3.48) also increase the probability relative to their baseline categories, AMD and Asus. This makes sense, as Nvidia GPUs are often connected to higher-performance laptops, while the positive HP and Dell effects suggest that laptops from these manufacturers are more likely to be high-priced than similar Asus laptops in this dataset.

In contrast, a larger **Screen** decreases the probability (coefficient = -1.195, OR = 0.30). This is somewhat counterintuitive since larger screens might be expected to increase price, but the data suggests the opposite after controlling for the other laptop characteristics.

Problem 4(e)

For which variables do the linear and logistic coefficients share a sign, and for which do they differ?

```
In [20]: signs = compare[["linear_coef", "logit_coef", "linear_p", "logit_p"]].copy()
signs["linear_sign"] = np.where(signs["linear_coef"] > 0, "+", "-")
signs["logit_sign"] = np.where(signs["logit_coef"] > 0, "+", "-")
signs["agreement"] = np.where(signs["linear_sign"] == signs["logit_sign"], "SAME", "DIFFERENT")
print(signs.round(4).to_string())
```

ent	linear_coef	logit_coef	linear_p	logit_p	linear_sign	logit_sign	agreem
Intercept	1434.2466	29.4861	0.0000	0.9961	+	+	S
AME							
C(Company) [T.Dell]	86.7254	1.2456	0.0438	0.0129	+	+	S
AME							
C(Company) [T.HP]	170.3954	1.4910	0.0001	0.0013	+	+	S
AME							
C(Company) [T.Lenovo]	35.3279	-0.0525	0.4020	0.9061	+	-	DIFFER
ENT							
C(TypeName) [T.Notebook]	-76.2954	-17.6031	0.1943	0.9976	-	-	S
AME							
C(TypeName) [T.Ultrabook]	416.3939	-16.1946	0.0000	0.9978	+	-	DIFFER
ENT							
C(GPU) [T.Intel]	165.4297	-0.1053	0.0000	0.7661	+	-	DIFFER
ENT							
C(GPU) [T.Nvidia]	218.1386	2.0148	0.0000	0.0009	+	+	S
AME							
Screen	-120.9323	-1.1954	0.0000	0.0001	-	-	S
AME							
Memory	87.3725	0.7421	0.0000	0.0000	+	+	S
AME							
Weight	271.0986	1.5361	0.0000	0.0889	+	+	S
AME							
Rating	-11.8206	-0.0904	0.0106	0.0660	-	-	S
AME							

Answer (e).

The coefficients have the **same sign** for **Company = Dell (+)**, **Company = HP (+)**, **TypeName = Notebook (-)**, **GPU = Nvidia (+)**, **Screen (-)**, **Memory (+)**, **Weight (+)**, and **Rating (-)**.

The signs are **different** for **Company = Lenovo** (+ in linear, - in logistic), **GPU = Intel** (+ in linear, - in logistic), and **TypeName = Ultrabook** (+ in linear, - in logistic). However, again, the **TypeName** logistic coefficient should be interpreted cautiously because of the quasi-complete separation identified in part (a).

Problem 4(f)

Probability that a Lenovo Ultrabook (Intel GPU), Screen = 8, Memory = 8, Weight = 4.2, Rating = 7 is high-priced.

$$P(\text{high} = 1 \mid x) = \frac{1}{1 + e^{-z}}, \quad z = \beta_0 + \beta_{\text{Lenovo}} + \beta_{\text{Ultrabook}} + \beta_{\text{Intel}} + \beta_{\text{Screen}}(8) + \beta_{\text{Memory}}(8) + \beta_{\text{Weight}}(4.2) + \beta_{\text{Rating}}(7)$$

The Dell/HP, Notebook and Nvidia indicators are all 0 for this laptop, so they drop out.

```
In [21]: new_laptop_4 = pd.DataFrame({
    "InventoryID": [4096],
    "Company":     ["Lenovo"],
    "TypeName":    ["Ultrabook"],
    "GPU":         ["Intel"],
    "Screen":      [8.0],
    "Memory":      [8],
    "Weight":      [4.2],
    "Rating":      [7],
})

b = log_model.params
terms = {
    "Intercept":          b["Intercept"],
    "Company=Lenovo":      b["C(Company) [T.Lenovo]"],
    "TypeName=Ultrabook": b["C(TypeName) [T.Ultrabook]"],
    "GPU=Intel":           b["C(GPU) [T.Intel]"],
    "Screen x 8.0":        b["Screen"] * 8.0,
    "Memory x 8":          b["Memory"] * 8,
    "Weight x 4.2":        b["Weight"] * 4.2,
    "Rating x 7":          b["Rating"] * 7,
}

for k, v in terms.items():
    print(f" {k:<20} = {v:>12.4f}")

z_val = sum(terms.values())
p_manual = 1 / (1 + np.exp(-z_val))
p_smf = log_model.predict(new_laptop_4).iloc[0]
```

```
print(f"\n z (log-odds)           = {z_val:.4f}")
print(f"  P(high=1) manual         = {p_manual:.8f}")
print(f"  P(high=1) via .predict      = {p_smf:.8f}")
print(f"  1 - P(high=1)                = {1 - p_manual:.3e}")
```

```
Intercept           = 29.4861
Company=Lenovo       = -0.0525
TypeName=Ultrabook   = -16.1946
GPU=Intel            = -0.1053
Screen x 8.0         = -9.5635
Memory x 8           = 5.9365
Weight x 4.2         = 6.4516
Rating x 7           = -0.6325
```

```
z (log-odds)        = 15.3257
P(high=1) manual     = 0.99999978
P(high=1) via .predict = 0.99999978
1 - P(high=1)       = 2.209e-07
```

Answer (f).

For this laptop, the fitted log-odds are

$$z = 29.4861 - 0.0525 - 16.1946 - 0.1053 - 1.1954(8) + 0.7421(8) + 1.5361(4.2) - 0.0904(7) \approx 15.33.$$

Therefore,

$$P(\text{high} = 1) = \frac{1}{1 + e^{-15.33}} \approx 0.99999978.$$

Thus, the fitted model predicts an approximately **100% probability** that this laptop is high-priced. This prediction should be interpreted cautiously because the logistic regression had the quasi-complete separation issue identified in part (a).

Problem 4(g)

Apply the model to the test set with a 0.5 probability cutoff and compute accuracy.

```
In [22]: test_probs = log_model.predict(test)
test_class = (test_probs >= 0.5).astype(int)

conf = pd.crosstab(test["high"], test_class, rownames=["Actual"], colnames=["Predicted"])
print("Confusion matrix (test set):")
print(conf.to_string(), "\n")

n_correct = (test_class == test["high"]).sum()
accuracy = n_correct / len(test)
baseline = test["high"].mean() # naive rule: always predict "high"

print(f"Test accuracy (cutoff 0.5) = {accuracy:.4f} ({n_correct} / {len(test)})")
print(f"Baseline accuracy = {baseline:.4f}")
```

Confusion matrix (test set):

Predicted	0	1
Actual		
0	27	26
1	15	212

Test accuracy (cutoff 0.5) = 0.8536 (239 / 280)

Baseline accuracy = 0.8107

Answer (g).

Using a probability cutoff of 0.5, the model correctly classifies **239 out of 280** laptops in the test set. Thus, the test accuracy is

$$\text{Accuracy} = \frac{239}{280} = 0.8536 \approx 85.4\%.$$

This is slightly better than the **81.1% naive baseline** from always predicting a laptop as high-priced.